LEARN BEING FORWARD

Programming — Python Programming

Data structures — lists, dicts, sets

COMPLETE LEARNING GUIDE

What You'll Learn

Python's four built-in collection types — lists, tuples, dictionaries, and sets — cover the overwhelming majority of real-world data-shaping needs, and choosing the right one for a given problem (ordered and mutable vs. immutable vs. unique-only vs. key-lookup) is one of the most practically useful skills in the language. This sub-module covers each structure's core operations and performance characteristics, the list/dict/set comprehension syntax that makes Python code notably more concise than equivalent loop-based code in many other languages, and the shallow-vs-deep copy distinction that trips up even experienced developers once nested structures are involved. By the end, choosing between a list, a set, and a dict for a new piece of code should be an immediate, almost reflexive decision based on the access pattern needed.

1. Lists: Ordered, Mutable Sequences

A list is an ordered, mutable, heterogeneous collection — index-based access (lst[i]) and slicing (lst[a:b]) are O(1) and O(k) respectively, while searching for a value with `in` is O(n) since it scans sequentially. Lists support in-place mutation (append, insert, remove, pop, sort), which matters because several of these return None rather than a new list — calling `sorted_list = my_list.sort()` is a classic bug, since .sort() mutates in place and returns None.

- list.append(x) adds to the end in O(1) amortized time; list.insert(0, x) is O(n) since every later element must shift.
- .sort() mutates in place and returns None; the built-in sorted() returns a new sorted list and leaves the original untouched.

PYTHON

```
numbers = [5, 2, 8, 1]
numbers.append(9)          # [5, 2, 8, 1, 9]
numbers.sort()             # mutates in place: [1, 2, 5, 8, 9]

original = [3, 1, 2]
new_list = sorted(original)  # original stays [3, 1, 2]; new_list is [1, 2, 3]
```

2. List Comprehensions

A list comprehension — [expression for item in iterable if condition] — builds a new list in a single, readable line and is generally both faster and more idiomatic than the equivalent manual loop with .append() calls. Comprehensions can nest, filter with an `if` clause, and even include a conditional expression that changes the mapped value itself (not just whether it's included), though deeply nested comprehensions should usually be rewritten as a plain loop once they stop being readable at a glance.

- [x for x in range(10) if x % 2 == 0] filters; [x if x % 2 == 0 else -x for x in range(10)] transforms every element conditionally — these are different patterns and easy to confuse.

```python
squares = [n ** 2 for n in range(10)]
even_squares = [n ** 2 for n in range(10) if n % 2 == 0]
flattened = [x for row in [[1, 2], [3, 4], [5]] for x in row]

print(even_squares)  # [0, 4, 16, 36, 64]
print(flattened)     # [1, 2, 3, 4, 5]
```

3. Tuples: Immutable, Ordered Sequences

A tuple looks like a list but cannot be modified after creation — no append, no item assignment. That immutability makes tuples hashable (when their contents are also hashable), so they can be used as dictionary keys or set members, which plain lists cannot. Tuples are also the natural return type for a function returning multiple values, since `return a, b` implicitly packs a tuple, and the caller can unpack it directly (`x, y = get_point()`).

- A single-element tuple needs a trailing comma: (5,) is a tuple, but (5) is just the int 5 in parentheses.
- Because tuples are hashable (if their elements are), they can be dict keys — lists never can be, since lists are mutable.

```python
def min_max(values: list[int]) -> tuple[int, int]:
    return min(values), max(values)

low, high = min_max([4, 1, 9, 2])
print(low, high)  # 1 9

distances = {(0, 0): 0, (1, 1): 1.41}  # tuples as dict keys
```

4. Dictionaries: Key-Value Mappings

A dict maps unique, hashable keys to values with average O(1) lookup, insertion, and deletion — by far the most common structure for 'look something up by name/id' problems. Since Python 3.7, dicts preserve insertion order as a language guarantee (not just an implementation detail). .get(key, default) avoids a KeyError for a missing key (plain dict[key] raises one), and dict comprehensions ({k: v for k, v in ...}) build dictionaries with the same conciseness as list comprehensions.

- dict[key] raises KeyError if the key is absent; .get(key, default) returns the default instead — prefer .get() when a missing key is an expected possibility.
- Iterating `for key, value in my_dict.items()` is the idiomatic way to get both the key and value together.

```python
inventory = {"apples": 50, "bananas": 30}
inventory["cherries"] = 15           # add a new key
print(inventory.get("grapes", 0))    # 0 — no KeyError

doubled = {k: v * 2 for k, v in inventory.items()}
print(doubled)  # {'apples': 100, 'bananas': 60, 'cherries': 30}
```

5. Sets: Unordered Collections of Unique Items

A set stores unique, hashable items with no guaranteed order and offers O(1) average membership testing (`x in my_set`) — far faster than the O(n) membership check on a list once the collection grows large. Sets directly implement mathematical set operations: union (|), intersection (&), difference (-), and symmetric difference (^), which makes them the right structure for deduplication and for comparing two collections for overlap or difference.

- set(my_list) is the standard idiom for deduplicating a list while discarding order — wrap it in list() again if order genuinely doesn't matter afterward.
- Sets cannot contain unhashable items (like lists or other sets) as elements, only hashable ones (numbers, strings, tuples of hashables).

```python
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a & b)   # {3, 4}        intersection
print(a | b)   # {1,2,3,4,5,6} union
print(a - b)   # {1, 2}        difference

deduped = list(set([1, 2, 2, 3, 3, 3]))  # order not guaranteed: e.g. [1, 2, 3]
```

6. Choosing the Right Structure

The choice usually comes down to the access pattern a problem actually needs: use a list when order matters and you'll mostly iterate or index sequentially; a tuple when the data is fixed-size and shouldn't change (coordinates, a database row); a dict when you need fast lookup by a meaningful key; and a set when you only care about membership and uniqueness, not order or associated values. Picking a list and then doing `if x in my_list` repeatedly inside a loop is a frequent and avoidable performance mistake — converting to a set first turns an $O(n^2)$ pattern into O(n).

- `if x in collection` is O(n) for a list/tuple but O(1) average for a set or dict's keys — a very common, very fixable performance bug.

7. Shallow Copy vs. Deep Copy

Assigning a_list = b_list copies the reference, not the data — both names point to the exact same list. list(b_list) or b_list.copy() creates a shallow copy: a new outer list, but any nested mutable objects inside it (a list of lists, for example) are still shared references to the same inner objects. To fully duplicate a nested structure so that no level shares memory with the original, copy.deepcopy() from the standard library's copy module recursively copies every nested level.

- A shallow copy is enough when the collection holds only immutable elements (ints, strings, tuples); once it holds nested lists/dicts, a shallow copy still shares those inner objects.

```python
import copy

original = [[1, 2], [3, 4]]
shallow = original.copy()
deep = copy.deepcopy(original)

shallow[0].append(99)   # mutates the SHARED inner list
print(original)         # [[1, 2, 99], [3, 4]] — original was affected too!

deep[1].append(100)
print(original)         # [[1, 2, 99], [3, 4]] — unaffected by the deep copy's change
```

Common Pitfalls

- Writing `my_list = my_list.sort()`, forgetting that .sort() mutates in place and returns None — the variable now holds None, not a sorted list.
- Using `dict[key]` when the key might legitimately be missing, crashing with a KeyError instead of using .get(key, default).
- Repeatedly checking `if x in my_list` inside a loop over a large list instead of converting to a set first for O(1) membership checks.
- Trying to use a list as a dictionary key or a set element, forgetting that only hashable (effectively immutable) types are allowed.
- Assuming `new = old.copy()` fully duplicates a nested structure — a shallow copy still shares any nested mutable objects with the original.
- Forgetting the trailing comma needed to make a single-element tuple: (5) is just the int 5, not a tuple.

Key Takeaways

- Lists: ordered, mutable, O(1) index access, O(n) membership check. Use when order matters and you mostly iterate/index.
- Tuples: ordered, immutable, hashable if contents are — the natural choice for fixed-size records and multi-value returns.
- Dicts: unique hashable keys to values, O(1) average lookup, insertion-ordered since Python 3.7 — the default choice for 'look up by key.'
- Sets: unique, unordered, O(1) average membership test, support union/intersection/difference directly — ideal for deduplication and overlap checks.
- List/dict/set comprehensions are the idiomatic, usually-faster alternative to building a collection with a manual loop and repeated .append()/assignment.
- Shallow copies share nested mutable objects with the original; use copy.deepcopy() when a fully independent nested structure is required.

Learn More

Python Docs — Data Structures
https://docs.python.org/3/tutorial/datastructures.html

Python Docs — copy: Shallow and Deep Copy Operations
https://docs.python.org/3/library/copy.html

Python Docs — Mapping Types (dict)
https://docs.python.org/3/library/stdtypes.html#mapping-types-dict